TP MI-GUIDÉ

Construction Progressive d'un Système de Boissons

Réalisé par :

Oualili Chayma

Encadré par :

Prof Sara AHSAIN

1. Description du problème

Ce TP a pour objectif de construire progressivement un système de gestion de boissons pour un café, en appliquant les principes de la Programmation Orientée Objet (POO) en Python.

Le système doit permettre de :

- Représenter des boissons avec un prix et une description
- Enrichir une boisson avec des ingrédients (lait, sucre, caramel…)
- Combiner plusieurs boissons entre elles
- Gérer des clients et leurs points de fidélité
- Traiter des commandes sur place ou à emporter

2. Architecture choisie

2.1 Classe abstraite Boisson

La classe Boisson (héritant de ABC) constitue le socle du système. Elle déclare deux méthodes abstraites que toute boisson concrète doit obligatoirement implémenter :

- cout() — retourne le prix de la boisson
- description() — retourne la description textuelle

La méthode __add__ est également définie dans cette classe pour permettre la combinaison de deux boissons via l'opérateur +.

2.2 Boissons concrètes

Deux classes héritent directement de Boisson :

- Cafe — prix : 2.00 DH, description : "Café simple"
- The — prix : 1.50 DH, description : "Thé"

2.3 Patron Décorateur (Design Pattern)

Pour éviter l'explosion combinatoire de classes (CafeLait, CafeSucre, CafeLaitSucre…), le patron de conception Décorateur est utilisé. La classe DecorateurBoisson enveloppe une boisson existante et lui ajoute un comportement supplémentaire.

Ingrédients disponibles :

- Lait — +0.50 DH
- Sucre — +0.20 DH
- Caramel — +0.70 DH

Chaque décorateur délègue l'appel à la boisson qu'il enveloppe, puis ajoute sa contribution au prix et à la description.

2.4 Dataclass Client

La classe Client est déclarée avec le décorateur @dataclass, ce qui génère automatiquement __init__, __repr__ et __eq__. Elle contient :

- nom (str)
- numero (int)
- points_fidelite (int, défaut : 0)

2.5 Hiérarchie des commandes

La classe Commande gère l'association client-boissons et le calcul du prix total. Deux sous-classes en héritent pour personnaliser l'affichage :

- CommandeSurPlace — affichage avec en-tête "COMMANDE SUR PLACE"
- CommandeEmporter — affichage avec message de récupération au comptoir

2.6 Héritage multiple : CommandeFidele

La classe CommandeFidele hérite à la fois de Commande et de Fidelite. Lors de la validation d'une commande, elle attribue automatiquement des points de fidélité au client (1 point par Dirham dépensé).

2.7 Schéma des relations

Boisson (ABC)

```
├── Cafe
├── The
└── DecorateurBoisson
    ├── Lait
    ├── Sucre
    └── Caramel
```

Commande

```
├── CommandeSurPlace
├── CommandeEmporter
└── CommandeFidele  (hérite aussi de Fidelite)
```

3. Principales fonctionnalités implémentées

1. Classe abstraite Boisson avec méthodes obligatoires cout() et description()
2. Boissons concrètes Cafe et The
3. Décorateurs Lait, Sucre, Caramel empilables à l'infini
4. Combinaison de boissons via l'opérateur + (__add__)
5. Dataclass Client avec points de fidélité
6. Classe Commande avec ajout de boissons et calcul du total
7. Sous-classes CommandeSurPlace et CommandeEmporter avec affichage personnalisé
8. Classe Fidelite et héritage multiple dans CommandeFidele
9. Attribution automatique de points lors de la validation d'une commande

4. Exemples d'exécution

4.1 Partie 6 — Café avec Lait et Sucre

Code :

```
boisson = Cafe()
boisson = Lait(boisson)
boisson = Sucre(boisson)
print(boisson.description())
print(boisson.cout())
```

Résultat :

```
Commande : Café simple, Lait, Sucre
Prix : 2.7 DH
```

4.2 Partie 7 — Commande complète avec fidélité

Code :

```
chayma= Client(nom='Chayma', numero=42)
cafe_caramel = Caramel(Lait(Cafe()))
the_sucre    = Sucre(The())
commande = CommandeFidele(alice)
commande.ajouter_boisson(cafe_caramel)
commande.ajouter_boisson(the_sucre)
commande.afficher()
commande.valider()
```

Résultat :

```
Client       : Chayma (n°42)
Commande   : Café simple, Lait, Caramel
Prix       : 3.20DH
Commande   : Thé, Sucre
Prix       : 1.70DH
Total        : 4.90DH

[Validation de la commande — montant : 4.90DH]
  +4 point(s) de fidélité ajouté(s) → Total : 4 point(s)
```

4.3 Commande à emporter

Résultat :

```
===== COMMANDE À EMPORTER =====
Client       : Manal (n°7)
Commande   : Café simple, Caramel
Prix       : 2.70DH
Total        : 2.70DH
  [Veuillez passer au comptoir pour récupérer votre commande]
================================
```

5. Questions de réflexion (Partie 8)

Question 1 — Ajout de nouveaux ingrédients

La classe DecorateurBoisson permet d'ajouter facilement de nouveaux ingrédients. Il suffit de créer une nouvelle sous-classe héritant de DecorateurBoisson et d'y définir cout() et description(). Aucune classe existante n'est modifiée .

Question 2 — Ajout d'une nouvelle boisson

Si l'on souhaitait ajouter une boisson "Chocolat chaud", il suffirait de créer une nouvelle classe ChcolatChaud(Boisson) avec ses propres implémentations de cout() et description(). Le reste du système (décorateurs, commandes, fidélité) fonctionnerait immédiatement avec cette nouvelle boisson, sans aucune modification.

Question 3 — Séparation des responsabilités

Séparer les responsabilités entre plusieurs classes (principe SRP — Single Responsibility Principle) rend le programme plus facile à maintenir car :

- Chaque classe a un rôle unique et bien défini
- Modifier la logique de fidélité n'impacte pas les boissons
- Ajouter un type de commande n'impacte pas les ingrédients
- Le code est plus lisible, testable et extensible sans risque de régression